DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute Affiliated to VTU, Belagavi, Accredited by NAAC with 'A' Grade)

Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111

Department of Artificial Intelligence & Machine Learning

FULL STACK DEVELOPMENT

LABORATORY MANUAL

V Semester
Course Code: 21AIL584
Course Type: LABORATORY

Scheme: 2021-2025

Version 1.1

w.e.f. November ,2023

Editorial Committee

Full Stack Development Laboratory Faculty, Dept. of AI&ML

Approved by

Dr. Vindhya P Malagi, H.O.D

Document Log

Name of the Document	Full Stack Development Laboratory Manual
Syllabus Scheme	2021-2025
Current Version and Date	Version 1.1, Nov 2023
Subject Code	21AIL584
Editorial Committee	Full Stack Development Laboratory Faculty
Approved by	HOD, Dept. of A I & M L
Lab Faculty	Dr. Reshma S Dr. Aruna N G Prof. Deepashree
Lab Instructor	Mr. Kumar

Table of Contents

Vision of the Department

To provide progressive education and flourish the student's ingenuity to be successful professionals impacting the society for a smarter and ethical world.

Mission of the Department

M1: *To adopt an engaging teaching learning process with emphasis on problem solving and programming skills.*

M2: *To promote additional skill development through enhanced and experiential learning.*

M3: *To collaborate with industries and professional bodies and make the students industry ready.*

M4: *To encourage innovation through multi-disciplinary research and development activities.*

M5: *To imbibe human values and ethics in students to make them impact the society's as responsible professionals.*

Program Educational Objectives (PEOs) of Department

After the course completion, AI&ML graduates will be able to:

PEO1: To practice and implement their success skills of problem solving, communication. and collaboration for providing innovative engineering solutions.

PEO2: Contribute their AI & ML expertise grounded in computer science and mathematics as members and leaders of professional engineering teams in multidisciplinary domains.

PEO3: Demonstrate lifelong learning through continued professional development and higher education in top graduate technical, research and management programs.

PEO4: Demonstrate a commitment to society by applying the skills and knowledge for a smarter and ethical world.

Program Specific Outcomes (PSOs)

PSO1: Graduates will be adept in programming and problem-solving skills for developing and managing software.

PSO2: Graduates will be able to identify, formulate , predict , and solve real world Problems by applying principles of Artificial Intelligence & Machine Learning.

PSO3: Graduates will be proficient to efficiently manage computer system resources. using cloud computing technologies.

Program Outcomes (POs)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. **Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. **Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

SYLLABUS

Course code : 21AIL584 **Credits: 01**

L: P: T: S : 0:2:0:0 **CIE Marks: 50**

Exam Hours: 03 **SEE Marks: 50**

Total Hours : 40

Course Objectives:

Learning the MERN (MongoDB , Express.js , React.js , Node.js) stack can be an excellent way to build full-stack web applications. Here are four objectives.

- MongoDB: Learn the basics of MongoDB, a NoSQL database, including how to install, configure, and interact with it using the MongoDB shell and Compass GUI.
- Express.js : Gain an understanding of Express.js, a web application framework for Node.js, focusing on routing, middleware, and handling HTTP requests and responses.
- React.js : Dive into React.js, a JavaScript library for building user interfaces, learning about components, state management, props, hooks, and routing.
- Node.js: Explore Node.js, a JavaScript runtime environment , understanding its event-driven architecture, asynchronous programming, modules, and package management using npm or yarn.

Course Outcomes: At the end of the course, student will be able to:

21AIL584.1	Applying JavaScript fundamentals to solve real-world problems.
21AIL584.2	Develop functional React applications.
21AIL584.3	Building Full-Stack Applications with ExpressJS, NodeJS, and MongoDB:
21AIL584.4	Developing complete web applications using the MERN Stack.

Mapping of Course outcomes to Program outcomes:

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	**3**				**3**				**3**			**3**
CO2	**3**				**3**				**3**			**3**
CO3	**3**	**3**			**3**				**3**			**3**
CO4	**3**	**3**			**3**				**3**			**3**

Module	Contents of the Module	Hours	COs
1	JavaScript Introduction to JavaScript: Applying JavaScript (internal and external), Understanding JS Syntax, Introduction to Document and Window Object, Variables and Operators, Data Types and Num Type Conversion, Math and String Manipulation, Objects and Arrays, Date and Time Conditional Statements, Switch Case, Looping in JS, Functions	08	CO1
2	ReactJS Introduction Templating using JSX, Components, State and Props, Lifecycle of, Components, Rendering List and Portals, Error Handling, Routers, Service Side Rendering, Unit Testing, Webpack	08	CO2
3	NodeJS Node js Overview: Node js - Basics and Setup, Node js Console, Node js Command Utilities, Node js Modules, Node js Concepts,Node js Events,Node js with Express js,Node js Database Access.	08	CO3, CO4
4	Express .js MVC Patter, Introduction to express, routing, HTTP Interaction, Handling form data, cookies & sessions, Restful services.	08	CO3, CO4
5	MongoDB SQL and NoSql Concepts: Create and Manage MongoDB, MongoDB Data Modeling and CRUD Operations, MongoDB with Exoress, MongoDB with NodeJS, Services Offered by MongoDB	08	CO3, CO4

Text Books:

1. The Road to React: Your journey to master plain yet pragmatic React.js by Robin Wieruch ,2023 edition
2. Beginning Node.js, Express & MongoDB Development by Greg Lim
3. "Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node" by Vasan Subramanian, Author: Vasan Subramanian, Publication Year: 2017, Publisher: Apress

Reference Books:

1. Web Technology: Theory and Practice by M. Srinivasan
2. Beginning React (incl. Redux and React Hooks) by Greg Lim (Author)
3. Learning AngularJS: A Guide to AngularJS Development 1st , Kindle Edition, by Ken Williamson

   "Learning React: Modern Patterns for Developing React Apps" by Alex Banks and Eve Porcello Authors: Alex Banks and Eve Porcello, Publication Year: 2017, Publisher: O'Reilly Media

4. "MongoDB: The Definitive Guide" by Kristina Chodorow and Shannon Bradshaw, Authors: Kristina Chodorow and Shannon Bradshaw, Publication Year: 2013 (2nd Edition), Publisher: O'Reilly Media

COURSES

1) https://www.udemy.com/course/react-nodejs-express-mongodb-the-mern-fullstack-guide/

2) https://www.coursera.org/specializations/full-stack-open

3) https://university.mongodb.com/

LAB EVALUATION PROCESS

CIE-Practical (Conduction) :30 Min 12

WEEK WISE EVALUATION OF EACH PROGRAM

ACTIVITY	MARKS
Execution [10]	20
Observation Book + Record	10
TOTAL	30

CIE-Practical (Test) :20 Min 08

INTERNAL ASSESSMENT EVALUATION (End of Semester)		
S.No	**ACTIVITY**	**MARKS**
01	Write-up	10
02	Conduction	35
03	Viva Voce	05
	Total	50 (Reduced to 20 min 08)

Total CIE-Practical :50 Min 20 (Reduced to 20 Min 08)

FINAL INTERNAL ASSESSMENT CALCULATION		
S.No	**ACTIVITY**	**MARKS**
01	Average of weeklyentries	30
02	Internal Assessment reduced to	20
	Total	50

Reduce overall lab marks from 50 to 20. The minimum marks to be secured in CIE of the LAB to appear for SEE of IPCC shall be the 8 marks [40% of maximum marks (20)].

CIE (CONTINUOUS INTERNAL EVALUATION) :

Continuous Evaluation done every week.

Rubrics	MARKS
Observation	5
Conduction	10
Record	5
Viva	5
Attendance	5
TOTAL	**25**

Rubrics for Continuous Internal Evaluation

Table 1: Rubrics for lab CIE Max Marks: 25 (excluding attendance)

Evaluation / Rubrics	Not Satisfactory (1-2)	Satisfactory (3)	Good (4)	Excellent (5)
Observation (Inferring to the programs in the lab & documenting it in observation) (05 M)	Incomplete programs and missing analysis part, late submission	Complete programs with comments however analysis and inference not presented.	Complete programs with comments including analysis, inference not presented appropriately. Completed on time	Complete programs with comments including analysis and inference provided and completed on time
Conduction (10 M)	Executed the program without understanding, rectified the errors and warnings with help, lack of conceptual understanding on the program.	Successfully executed the program with basic understanding of the program	Successfully executed the program with good understanding of the program (function level)	Successfully executed the program with detailed understanding of the program (line by line)
Records (05 M)	Incomplete programs and missing analysis part, late submission	Complete programs with comments however analysis and inference not presented.	Complete programs with comments including analysis, however inference not presented appropriately. Submitted on time.	Complete programs with comments including analysis and inference provided & submitted on time
Viva (05 M)	1 question out of 5 viva questions answered on a one-on-one basis	Basic understanding on the programs and 2 or 3 questions out of 5 viva questions answered on a one-on-one basis	Good understanding on the programs and 4 questions out of 5 viva questions answered on a one-on-one basis	Programs well understood and all 5 questions answered on a one-on-one basis
Score	Score 1 (S1)	Score 2 (S2)	Score 3 (S3)	Score 4 (S4)
Total Score				**S1+ S2+S3+S4**

In the conduction criteria of CIE (both during internal and external lab exams), the following Rubrics are considered.

- Syntax
- Logic
- Correctness
- Completeness of the program

Table 2: Rubrics for lab programs conduction component

Max Marks:5

Rubrics	**Not Satisfactory (1-2)**	**Satisfactory (3)**	**Good (4)**	**Excellent(5)**
Syntax (5) Ability to understand and follow the rules of the program	Program doesn't compile and has errors and warnings	Program compiles, but contains warnings that lead to incorrect logic	Program compiles and it is free from errors but may contain non-standard usage of programming	Program compiles without errors and follows the standards of programming
Logic (5) Ability to identify conditions, control flow and data structures for the given program	Incorrect program logic such as infinite loops	Program logic is correct without any infinite loops, however all possible boundary cases not addressed	Program logic is correct with few boundary conditions	Program logic is correct with all conditions and cases coded with no ambiguity
Correctness (5) Ability to code algorithms that produce correct appropriate output for the given program	Incorrect output or inappropriate output for any given test case	Program gives correct output for most inputs but not for all test cases	Program gives correct output for most test cases	Program gives correct output for all test cases
Completeness (5) Ability to apply critical analysis to check for the completeness of the program	Program shows little or no recognition on different cases leading to incompleteness	Program shows evidence of case analysis and missing significant cases or missed cases	Most of the requirements of the program met however all possible cases not handled	All requirements of the program met and all possible cases handled
Total				**Score /4**

Rubrics for write up and execution of Internal Examination

TOPIC	MARKS
Write up	8
Execution	35
Viva	7
TOTAL	25

Max Marks:5

	Rubrics	Not Satisfactory (1-2)	Satisfactory (3)	Good (4-5)	Excellent (6-8)
WRITE-UP	**Syntax (8)** Ability to understand and follow the rules of the program.	Program doesn't compile and has errors and warnings	Program compiles, but contains warnings that lead to incorrect logic	Program compiles and it is free from errors but may contain non-standard usage of programming	Program compiles without errors and follows the standards of programming
	Logic (8) Ability to identify conditions, control flow and data structures for the given program	Incorrect program logic such as infinite loops	Program logic is correct without any infinite loops, however all possible boundary cases not addressed	Program logic is correct with few boundary conditions	Program logic is correct with all conditions and cases coded with no ambiguity
	Total				**Score /2**

Max Marks:35

	Rubrics	Not Satisfactory (1-2)	Satisfactory (3-4)	Good (5-6)	Excellent (7)
EXECUTION	**Correctness (7)** Ability to code algorithms that produce correct appropriate output.	Incorrect output or inappropriate output for any given test case	Program gives correct output for most inputs but not for all test cases	Program gives correct output for most test cases	Program gives correct output for all test cases
	Completeness (7) Ability to apply critical analysis to check for the completeness of the program	Program shows little or no recognition on different cases leading to incompleteness	Program shows evidence of case analysis and missing significant cases or missed cases	Most of the requirements of the program met however all cases not handled	All requirements of the program met and all possible cases handled
	Total				**Score*2.5**

SEE LAB EXAMINATION:

With Mini Project Component

TOPIC	MARKS
Mini Project	20
Write up	5
Execution	20
Viva	5
TOTAL	50

Table 3: Breakup of marks for SEE with mini project

Max Marks:05

	Rubrics	Not Satisfactory (1)	Satisfactory (2-3)	Good (4)	Excellent (5)
WRITE-UP	**Syntax (5)** Ability to understand and follow the rules of the program	Program doesn't compile and has errors and warnings	Program compiles, but contains warnings that lead to incorrect logic	Program compiles & it is free from errors but may contain non-standard programming	Program compiles without errors and follows the standards of programming
	Logic (5) Ability to identify conditions, control flow and data structures for the given program	Incorrect program logic such as infinite loops	Program logic is correct without any infinite loops, however all possible boundary cases not addressed	Program logic is correct with few boundary conditions	Program logic is correct with all conditions and cases coded with no ambiguity
	Total				**Score /2**

Max Marks:20

	Rubrics	Not Satisfactory (1-2)	Satisfactory (3)	Good (4)	Excellent (5)
EXECUTION	**Correctness (5)** Ability to code algorithms that produce correct appropriate output for the given program	Incorrect output or inappropriate output for any given test case	Program gives correct output for most inputs but not for all test cases	Program gives correct output for most test cases	Program gives correct output for all test cases
	Completeness (5) Ability to apply critical analysis to check for the completeness of the program	Program shows little or no recognition on different cases leading to incompleteness	Program shows evidence of case analysis and missing significant cases or missed cases	Most of the requirements of the program met however all possible cases not handled	All requirements of the program met and all possible cases handled
	Total				**Score*2**

CHAPTER 1

INTRODUCTION TO FULL STACK DEVELOPMENT

MERN stack is a collection of technologies that enables faster application development. It is used by developers worldwide. The main purpose of using MERN stack is to develop apps using JavaScript only. MERN Stack is a compilation of four different technologies that work together to develop dynamic web apps and websites. MERN stands for MongoDB, Express, React, Node, after the four key technologies that make up the stack.

- MongoDB — document database.
- Express(.js) — Node.js web framework.
- React(.js) — a client-side JavaScript framework.
- Node(.js) — the premier JavaScript web server

MongoDB

MongoDB is a leading NoSQL database solution renowned for its flexibility and scalability. It stores data in a document-oriented format, using JSON-like documents with dynamic schemas. This schema-less approach allows developers to swiftly iterate and adapt their data models without the constraints of a predefined schema. MongoDB excels in handling unstructured or semi-structured data, making it suitable for a wide array of applications, from content management systems to real-time analytics platforms. Its robust query language and aggregation framework empower developers to perform complex data manipulations efficiently. Additionally, MongoDB's distributed architecture supports horizontal scaling, enabling seamless scalability to meet the demands of growing datasets and high-throughput applications. Overall, MongoDB stands as a versatile and powerful choice for modern data storage needs. Book Store Application

Working: MongoDB operates as a server process, interacting with clients via various drivers and interfaces. Developers perform operations such as insertion, querying, updating, and deletion of documents using MongoDB's query language and aggregation framework. The database stores data in collections of JSON-like documents, providing flexibility and scalability. MongoDB's distributed architecture supports horizontal scaling for efficient handling of large datasets. Its rich feature set and robust performance make it a popular choice for modern web applications and data-driven solutions.

Express.js

Express.js is a minimalist and flexible web application framework for Node.js, designed for building robust and scalable web applications and APIs. It simplifies the process of handling HTTP requests and responses by providing a rich set of features, including routing, middleware support, and template rendering. Express.js allows developers to define routes for different URLs and HTTP methods, making it easy to create RESTful APIs and dynamic web applications. Its lightweight and unopinionated nature enable developers to customize their applications according to specific requirements, while its extensive ecosystem of middleware and plugins facilitate rapid development and extensibility. Overall, Express.js empowers developers to create efficient and maintainable web applications with ease.

Working: Express.js simplifies web application development by providing a robust framework for handling HTTP requests and responses. Developers define routes for different URLs and HTTP methods, directing requests to corresponding handler functions. Middleware functions enhance functionality by performing tasks such as parsing request bodies and handling authentication. Express.js supports template rendering for dynamic content generation and facilitates the creation of RESTful APIs. Its lightweight nature and extensive ecosystem of middleware enable rapid development and customization. Overall, Express.js streamlines web development, empowering developers to build scalable and efficient applications with ease. Book Store Application

React.js

React.js, commonly known as React, is a JavaScript library for building user interfaces, primarily developed and maintained by Facebook. It revolutionized front-end development by introducing a component-based architecture, enabling developers to create reusable and composable UI components. React utilizes a virtual DOM for efficient rendering, updating only the components that have changed, resulting in improved performance and faster user interactions. Its declarative syntax allows developers to describe how the UI should look at any given time, abstracting away the complexities of DOM manipulation. React promotes uni-directional data flow and embraces a functional programming paradigm, enabling developers to write cleaner and more maintainable code. With its thriving ecosystem, including tools like React Router and Redux, React has become a popular choice for building dynamic, interactive, and scalable web applications.

Working: In React.js, developers create reusable UI components that manage their state and render based on that state. Components are composed together to form complex user interfaces. When the state of a component changes, React efficiently updates the affected parts of the virtual DOM and re-renders the UI. React uses JSX, a syntax extension that allows embedding HTML-like syntax within JavaScript code. Developers use React's lifecycle methods to hook into component events, such as mounting, updating, and unmounting. Finally, React's one-way data binding ensures that changes to the state of a component propagate downwards to child components, maintaining a clear and predictable flow of data throughout the application.

Node.js

Node.js is a powerful JavaScript runtime environment built on Chrome's V8 JavaScript engine. It enables developers to run JavaScript code outside of a web browser, making it well-suited for server-side development. Node.js employs an event-driven, non-blocking I/O model, allowing for efficient handling of concurrent connections and high-throughput applications. It boasts a Book Store Application vast ecosystem of modules and libraries available through npm, the Node.js package manager, facilitating rapid development and extending its functionality. Node.js excels in building scalable network applications, such as web servers, APIs, and real-time chat applications, thanks to its lightweight and efficient architecture. With its versatility and robust performance, Node.js has become a popular choice for building modern, data-intensive applications.

Working: In Node.js, developers write server-side JavaScript code to build scalable and efficient network applications. Node.js uses an event-driven, non-blocking I/O model, allowing it to handle concurrent connections without blocking the execution of other code. Developers can leverage the vast ecosystem of npm packages to add functionality to their applications. Node.js applications are typically run using the command line interface, with the node command followed by the filename of the script to execute. Asynchronous programming techniques, such as callbacks, promises, and async/await, are commonly used to handle asynchronous operations efficiently. Node.js enables developers to create web servers, APIs, and various types of network applications with ease and flexibility.

Week-1-2. Write a program to create a website using HTML CSS and JavaScript?
1. Create a form and validate the contents of the form using Javascript.

AIM: To create a form and validate the contents of the form using Javascript.

Program:

Index.html

```
<!DOCTYPE html>
<html>
<head>
<script>
function validateForm() {
  let x = document.forms["myForm"]["fname"].value;
  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}
</script>
</head>
<body>
<h2>JavaScript Validation</h2>
<form name="myForm" onsubmit="return validateForm()"
method="post">
  Name: <input type="text" name="fname">
  <input type="submit" value="Submit">
</form>
</body>
</html>
```

OUTPUT:

Week-3. Write a program to build a Chat module using HTML CSS and JavaScript?

```
<div class="chat-box">
  <header>
    <h1>André F. Martins</h1>
    <span style="flex:1"></span>
    <span class="close-button">X</span>
  </header>
  <section id="message-list">
  </section>
  <footer>
    <input id="message-input" type="text" placeholder="Type a message...">
  </footer>
</div>

box-sizing: border-box

::selection
  background: #fd0
  color: #222

br
  font-size: 0

body
  font-family: "Karla", sans-serif
  margin: 0
  display: flex
  align-items: center
  justify-content: center
  width: 100vw
  height: 100vh
  font-size: 16px
  background-color: #ddd

.chat-box
  width: 300px
  height: 400px
  background-color: #eee
  border-radius: 16px 16px 0 0
  overflow: hidden
  border: 2px #45f solid

  header
    background-color: #45f
    color: #fff
```

```
  display: flex
  padding: 0 16px

  > *
    line-height: 40px

  h1
    margin: 0
    font-size: 1.1em

  .close-button
    cursor: pointer
    user-select: none

section
  overflow-y: scroll
  overflow-x: hidden
  height: calc(100% - 40px - 40px)
  padding: 12px

  div
    margin: 12px 0

  span
    display: inline-block
    padding: 6px 8px
    margin: 1px 0
    max-width: 90%
    word-wrap: break-word

  div.me
    text-align: left
    span
      background-color: #68c
      border-radius: 4px 16px 16px 4px
    span:first-of-type
      border-top-left-radius: 16px
    span:last-of-type
      border-bottom-left-radius: 16px


  div.not-me
    text-align: right
    span
      background-color: #bcf
      border-radius: 16px 4px 4px 16px
```

```
    span:first-of-type
      border-top-right-radius: 16px
    span:last-of-type
      border-bottom-right-radius: 16px

  footer
    input
      font-family: "Karla", sans-serif
      outline: 0
      padding: 0 12px
      height: 40px
      width: 100%
      font-size: 1em

function sendMessage(message, itsMe) { // ...Mario
  var messageList = document.getElementById("message-list");

  var scrollToBottom = (messageList.scrollHeight - messageList.scrollTop -
messageList.clientHeight < 80);

  var lastMessage = messageList.children[messageList.children.length-1];

  var newMessage = document.createElement("span");
  newMessage.innerHTML = message;

  var className;

  if(itsMe)
  {
    className = "me";
    scrollToBottom = true;
  }
  else
  {
    className = "not-me";
  }

  if(lastMessage && lastMessage.classList.contains(className))
  {
    lastMessage.appendChild(document.createElement("br"));
    lastMessage.appendChild(newMessage);
  }
  else
  {
    var messageBlock = document.createElement("div");
    messageBlock.classList.add(className);
```

```
    messageBlock.appendChild(newMessage);
    messageList.appendChild(messageBlock);
  }

  if(scrollToBottom)
    messageList.scrollTop = messageList.scrollHeight;
}

var message = document.getElementById("message-input");
message.addEventListener("keypress", function(event) {
  var key = event.which || event.keyCode;
  if(key === 13 && this.value.trim() !== "")
  {
    sendMessage(this.value, true);
    this.value = "";
  }
});

sendMessage("Hello!", true);
sendMessage("Hey!", false);
sendMessage("How are you doing?", false);
sendMessage("I'm doing well.", true);
sendMessage("What about you?", true);
sendMessage("Good", false);

function sendRandomMessages()
{
  // http://www.conversationstarters.com/generator.phpvar
  messageList = [
    "What is your biggest concern about the future?",
    "What is your biggest fear?",
    "At what age would you consider someone to be old?",
    "What is your favorite home cooked dish?",
    "What is the biggest priority in your life right now?",
    "Where did you go on your last vacation?",
    "What was the biggest life change you've gone through?","Do
    you have any siblings?",
    "What's your family like?",
    "If you knew that you only had a year left to live, what would you do?",
    "What do you usually eat in the morning?",
    "What's in your fridge?",
    "Do you recycle?",
    "What are some things that you should not not say during a job interview?","If
    you could go back in time and change one thing, what would that be?", "What do
    you wear to sleep?",
    "What is the last thing you do before you go to sleep?",
```

```
    "If you could only eat one thing for the rest of your life, what would it be?",
    "Would you rather be homeless for a year or be in jail for a year?",
    "What do you do for fun?"
  ];
  var choosenMessage = messageList[Math.floor(Math.random() * messageList.length)];

  sendMessage(choosenMessage, false);

  setTimeout(sendRandomMessages, Math.random()*10000 + 7000);
}

sendRandomMessages();
```

Exercise Program: How To Create Chat Messages

```
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
body {
  margin: 0 auto;
  max-width: 800px;
  padding: 0 20px;
}

.container {
  border: 2px solid #dedede;
  background-color: #f1f1f1;
  border-radius: 5px;
  padding: 10px;
  margin: 10px 0;
}

.darker {
  border-color: #ccc;
  background-color: #ddd;
}

.container::after {
  content: "";
  clear: both;
  display: table;
}

.container img {
  float: left;
  max-width: 60px;
  width: 100%;
  margin-right: 20px;
  border-radius: 50%;
}

.container img.right {
  float: right;
  margin-left: 20px;
  margin-right:0;
}
```

```
.time-right  {
  float:  right;
  color: #aaa;
}

.time-left {
  float: left;
  color: #999;
}
</style>
</head>
<body>

<h2>Chat Messages</h2>

<div class="container">
  <img src="/w3images/bandmember.jpg" alt="Avatar" style="width:100%;">
  <p>Hello. How are you today?</p>
  <span class="time-right">11:00</span>
</div>

<div class="container darker">
  <img src="/w3images/avatar_g2.jpg" alt="Avatar" class="right" style="width:100%;">
  <p>Hey! I'm fine. Thanks for asking!</p>
  <span class="time-left">11:01</span>
</div>

<div class="container">
  <img src="/w3images/bandmember.jpg" alt="Avatar" style="width:100%;">
  <p>Sweet! So, what do you wanna do today?</p>
  <span class="time-right">11:02</span>
</div>

<div class="container darker">
  <img src="/w3images/avatar_g2.jpg" alt="Avatar" class="right" style="width:100%;">
  <p>Nah, I dunno. Play soccer.. or learn more coding perhaps?</p>
  <span class="time-left">11:05</span>
</div>

</body>
</html>
```

Week-4. Write a program to create a simple calculator Application using React JS

```
class App extends Component {
constructor() {
super()
this.state = { operations: [] }
}

}
render() {
    return (
<div className="App">
<Display data={this.state.operations} />
<Buttons>
<Button onClick={this.handleClick} label="C" value="clear" />
<Button onClick={this.handleClick} label="7" value="7" />
<Button onClick={this.handleClick} label="4" value="4" />
<Button onClick={this.handleClick} label="1" value="1" />
<Button onClick={this.handleClick} label="0" value="0" /><Button
onClick={this.handleClick} label="/" value="/" />
<Button onClick={this.handleClick} label="8" value="8" />
<Button onClick={this.handleClick} label="5" value="5" />
<Button onClick={this.handleClick} label="2" value="2" />
<Button onClick={this.handleClick} label="." value="." /><Button
onClick={this.handleClick} label="x" value="*" />
<Button onClick={this.handleClick} label="9" value="9" />
<Button onClick={this.handleClick} label="6" value="6" />
<Button onClick={this.handleClick} label="3" value="3" />
<Button label="" value="null" /><Button onClick={this.handleClick}
label="-" value="-" />
<Button onClick={this.handleClick} label="+" size="2" value="+" />
<Button onClick={this.handleClick} label="=" size="2" value="equal"
/>
</Buttons>
</div>
    )
class Buttons extends Component {
render() {
return <div className="Buttons"> {this.props.children} </div>
}
}class Button extends Component {
  render() {
    return (
<div
onClick={this.props.onClick}
className="Button"
        data-size={this.props.size}
        data-value={this.props.value} >
```

```
        {this.props.label}
</div>
    )
  }
}
class Display extends Component {
  render() {
    const string = this.props.data.join('')
    return <div className="Display"> {string} </div>
  }
}
handleClick = e => {
    const value = e.target.getAttribute('data-value')
    switch (value) {
      case 'clear':
this.setState({
          operations: [],
        })
        break
      case 'equal':
this.calculateOperations()
        break
      default:
        const newOperations = update(this.state.operations, {
          $push: [value],
        })
this.setState({
          operations: newOperations,
        })
        break
    }


calculateOperations = () => {
    let result = this.state.operations.join('')
    if (result) {
      result = math.eval(result)
      result = math.format(result, { precision: 14 })
      result = String(result)
this.setState({
        operations: [result],
      })
    }
  }
```

Week-5. Write a program to create a voting application using React JS

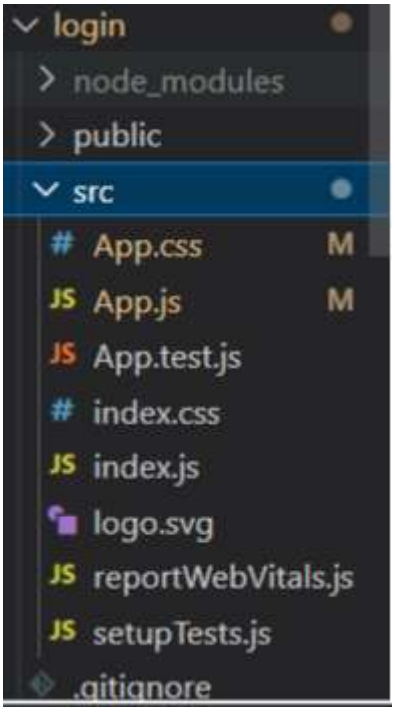
```
CREATE
ORREPLACEVIEW"public"."poll_results"AS
SELECT
  poll.id ASpoll_id,
o.option_id,
count(*)AS votes
FROM
(
(
SELECT
vote.option_id,
option.poll_id,
option.text
FROM
(
          vote
LEFTJOINoptionON((option.id =vote.option_id))
)
) o
LEFTJOIN poll ON((poll.id =o.poll_id))
)
GROUPBY
poll.question,
o.option_id,
  poll.id;
CREATE
ORREPLACEVIEW"public"."online_users"AS
SELECT
count(*)AS count
FROM
"user"
WHERE
(
"user".last_seen_at>(now()-'00:00:15' :: interval)
);
import{ApolloClient,HttpLink,InMemoryCache, split }from"@apollo/client";
import{GraphQLWsLink}from'@apollo/client/link/subscriptions';
import{createClient}from"graphql-ws";
import{getMainDefinition}from"@apollo/client/utilities";
constGRAPHQL_ENDPOINT="realtime-poll-example.hasura.app";

constscheme=(proto)=>
window.location.protocol==="https:"?`${proto}s`: proto;
```

```
constwsURI=`${scheme("ws")}://${GRAPHQL_ENDPOINT}/v1/graphql`;
consthttpURL=`${scheme("https")}://${GRAPHQL_ENDPOINT}/v1/graphql`;
constsplitter=({ query })=>{
const{ kind, operation }=getMainDefinition(query)||{};
constisSubscription=
    kind ==="OperationDefinition"&& operation ==="subscription";
returnisSubscription;};


const cache =newInMemoryCache();
const options ={ reconnect:true};

constwsLink=newGraphQLWsLink(createClient({ url:wsURI,connectionParams:{ options }}));
consthttpLink=newHttpLink({uri:httpURL});
const link =split(splitter,wsLink,httpLink);
const client =newApolloClient({ link, cache });
```

Week-8. Create a Simple Login form using React js

Now we are creating a login form which is a very important in any application or website as you know if open any website or application you will get a message to login if you click that you will be redirected to login page in this week, we will be creating login page

File Structure: -



----------**App.js**----------

```
import './App.css';

import React, { useState } from "react";

import ReactDOM from "react-dom";


function App() {

  // React States

  const [errorMessages, setErrorMessages] = useState({});

  const [isSubmitted, setIsSubmitted] = useState(false);
```

```
// User Login info
const database = [
 {
  username: "user1",
  password: "pass1"
 },
 {
  username: "user2",
  password: "pass2"
 }
];

const errors = {
 uname: "invalid username",
 pass: "invalid password"
};

consthandleSubmit = (event) => {
 //Prevent page reload
 event.preventDefault();

 var { uname, pass } = document.forms[0];

 // Find user login info
 constuserData = database.find((user) =>user.username === uname.value);
```

```
  // Compare user info
  if (userData) {
    if (userData.password !== pass.value) {
      // Invalid password
      setErrorMessages({ name: "pass", message: errors.pass });
    } else {
      setIsSubmitted(true);
    }
  } else {
    // Username not found
    setErrorMessages({ name: "uname", message: errors.uname });
  }
};

// Generate JSX code for error message
 constrenderErrorMessage = (name) =>
  name === errorMessages.name && (
    <div className="error">{errorMessages.message}</div>
  );

// JSX code for login form
constrenderForm = (
  <div className="form">
    <form onSubmit={handleSubmit}>
```

```
      <div className="input-container">
        <label>Username </label>
        <input type="text" name="uname" required />
        {renderErrorMessage("uname")}
      </div>
      <div className="input-container">
        <label>Password </label>
        <input type="password" name="pass" required />
        {renderErrorMessage("pass")}
      </div>
      <div className="button-container">
        <input type="submit" />
      </div>
    </form>
  </div>
 );

 return (
  <div className="app">
    <div className="login-form">
      <div className="title">Sign In</div>
      {isSubmitted ?<div>User is successfully logged in</div> :renderForm}
    </div>
  </div>
 );
                }
export default App;
```

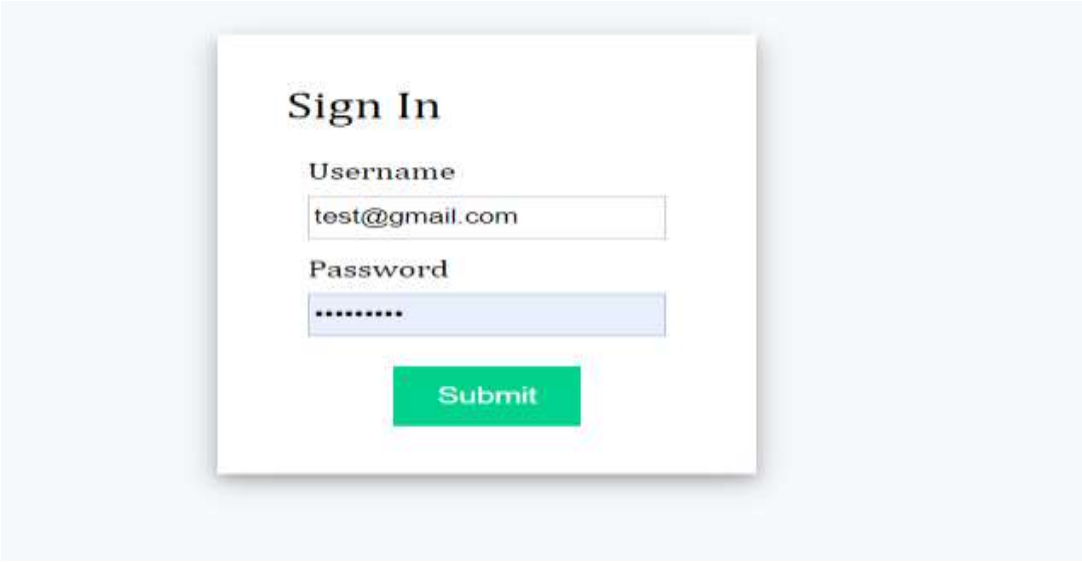
----------**Now Create a App.css file in same folder**----------

App.css

```
.app {
  font-family: sans-serif;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-direction: column;
  gap: 20px;
  height: 100vh;
  font-family: Cambria, Cochin, Georgia, Times, "Times New Roman", serif;
  background-color: #f8f9fd;
}

input[type="text"],
input[type="password"] {
height: 25px;
  border: 1px solid rgba(0, 0, 0, 0.2);
}

input[type="submit"] {
  margin-top: 10px;
```

```
  cursor: pointer;
  font-size: 15px;
  background: #01d28e;
  border: 1px solid #01d28e;
  color: #fff;
  padding: 10px 20px;
}

input[type="submit"]:hover {
  background: #6cf0c2;
}

.button-container {
  display: flex;
  justify-content: center;
}

.login-form {
  background-color: white;
  padding: 2rem;
  box-shadow: 0 4px 8px 0 rgba(0, 0, 0, 0.2), 0 6px 20px 0 rgba(0, 0, 0, 0.19);
}

.list-container {
  display: flex;
```

```
}
.error {
  color: red;
  font-size: 12px;
}
.title {
  font-size: 25px;
  margin-bottom: 20px;
}
.input-container {
  display: flex;
  flex-direction: column;
  gap: 8px;
  margin: 10px;
}
```

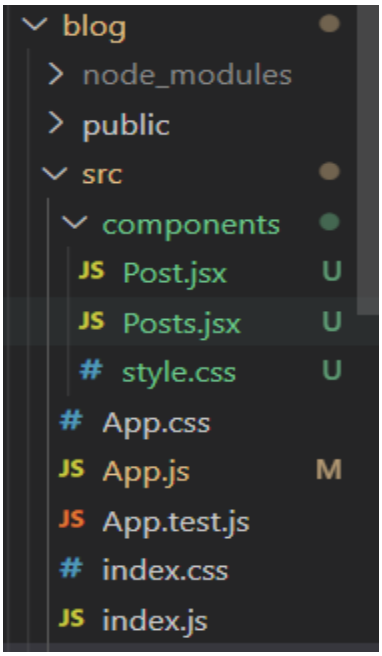


Week-9. Create a blog in React js

In this week we are going to create a blog website using react js

We have mainly 3 pages
1.App.js
2.Post.js
3.Posts.js

This is the Structure of the project



1.App.js

```
import logo from './logo.svg';
import './App.css';
import Posts from "./components/Posts";
function App() {
  return (
<div className="main-container">
<h1 className="main-heading">
       Blog App using React Js
</h1>
<Posts /></div>
);
}


export default App;
```

2. Posts.js

```
import React from "react";
import "./style.css";

import Post from "./Post";

const Posts = () => {
  const blogPosts = [
    {
      title: "JAVASCRIPT",
      body: `JavaScript is the world most popular
      lightweight, interpreted compiled programming
      language. It is also known as scripting
      language for web pages. It is well-known for
      the development of web pages, many non-browser
      environments also use it. JavaScript can be
      used for Client-side developments as well as
      Server-side developments`,
      author: "Nishant Singh ",
imgUrl:
        "https://media.geeksforgeeks.org/img-practice/banner/diving-into-excel-thumbnail.png",
    },
    {
      title: "Data Structure ",
      body: `There are many real-life examples of
      a stack. Consider an example of plates stacked
      over one another in the canteen. The plate
      which is at the top is the first one to be
      removed, i.e. the plate which has been placed
      at the bottommost position remains in the
      stack for the longest period of time. So, it
      can be simply seen to follow LIFO(Last In
      First Out)/FILO(First In Last Out) order.`,
      author: "Suresh Kr",
imgUrl:
        "https://media.geeksforgeeks.org/img-practice/banner/coa-gate-2022-thumbnail.png",
    },
    { title: "Algorithm",
    body: `The word Algorithm means “a process or set of rules to be followed in calculations
    or other problem-solving operations”. Therefore Algorithm refers to a set of rules/instructions that
    step-by-step define how a work is to be executed upon in order to get the expected results. `,
    author: "Monu Kr", imgUrl:
    "https://media.geeksforgeeks.org/img-practice/banner/google-test-series-thumbnail.png",
    },
```

```
    {
      title: "Computer Network",
      body: `An interconnection of multiple devices,
      also known as hosts, that are connected using
      multiple paths for the purpose of sending/
      receiving data media. Computer networks can
      also include multiple devices/mediums which
      help in the communication between two different
      devices; these are known as Network devices and
      include things such as routers, switches, hubs, and
      bridges. `,
      author: "Sonu Kr",
imgUrl:
        "https://media.geeksforgeeks.org/img-practice/banner/cp-maths-java-thumbnail.png",
    },
  ];

  return (
<div className="posts-container">
      {blogPosts.map((post, index) => (
<Post key={index} index={index} post={post} />
      ))}
</div>
  );
};

export default Posts;
```

1.Post.js

```
import React from "react";
import "./style.css";
const Post = ({ post: { title, body,
imgUrl, author }, index }) =>
{ return (
<div className="post-container">
<h1 className="heading">{title}</h1>
<imgclassName="image" src={imgUrl} alt="post" />
<p>{body}</p>
<div className="info">
<h4>Written by: {author}</h4>
</div>
</div>
);
}; export default Post;
```

Now we will style the project

Style.css in componenets folder

```
body {
    background-color: #0e9d57;
}
.posts-container {
    display: flex;
    justify-content: center;
    align-items: center;
}
.post-container {
    background: #e2e8d5;
    display: flex;
    flex-direction: column;
    padding: 3%;
    margin: 0 2%;
    height: 40%;
}
.heading {
    height: 126px;
    text-align: center;
    display: flex;
    align-items: center;
}
.image {
    width: 100%;
    height: 210px;
}
```

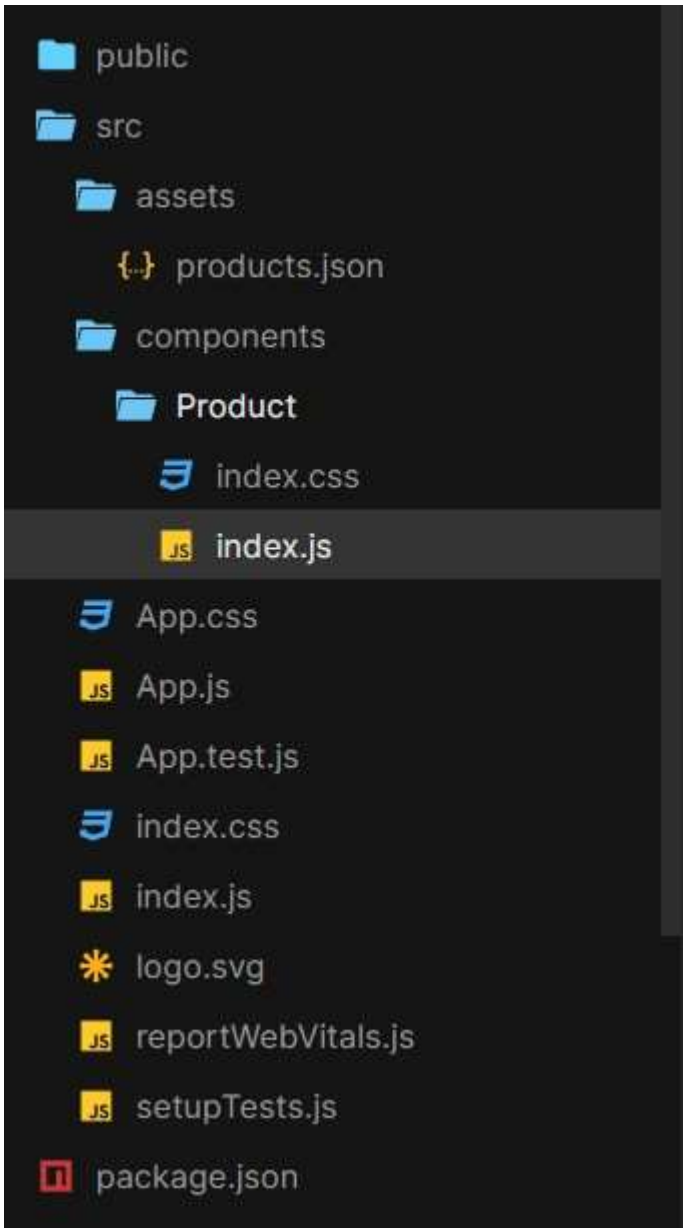
OUTPUT

Week-10. Create a project on Grocery delivery application

Assume this project is for a huge online departmental store. Assume that they have a myriadof grocery items at their godown. All items must be listed on the website, along with theirquantities and prices.Users must be able to sign up and purchase groceries. The systemshould present him with delivery slot options, and the user must be able to choose hispreferred slot. Users must then be taken to the payment page where he makes the payment with his favorite method.

This week will have many pages like Header,footer,categories and app.jsx

File Structure:

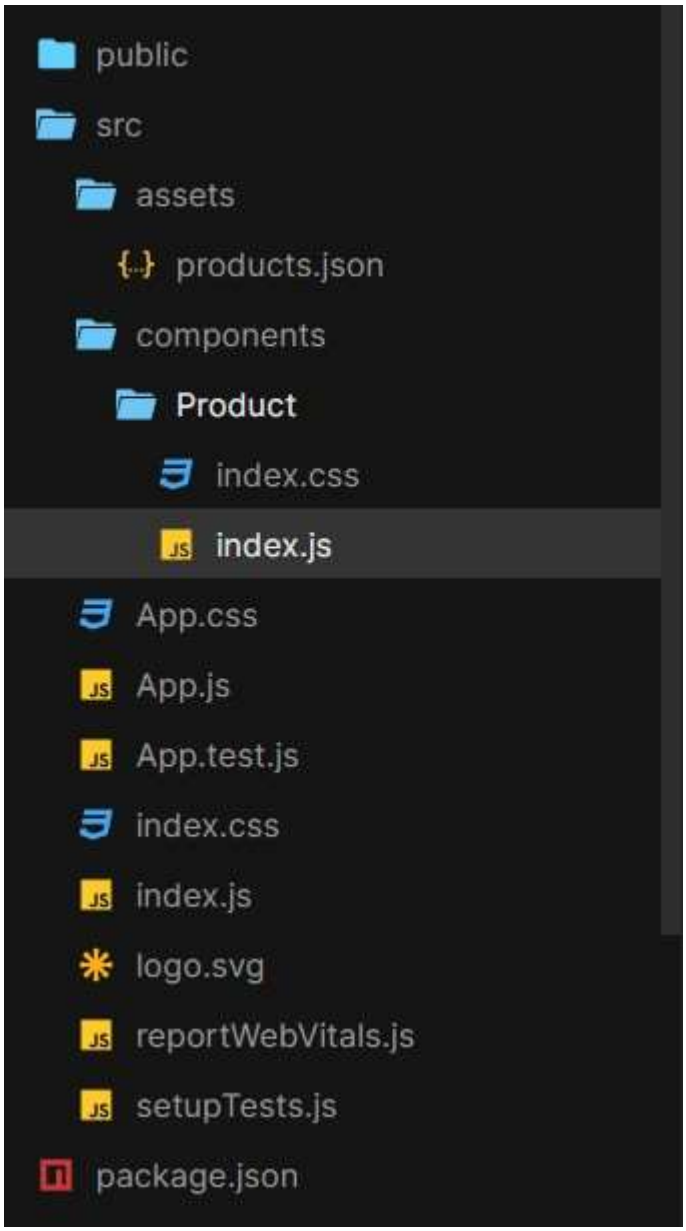


Week-10. Create a project on Grocery delivery application

Assume this project is for a huge online departmental store. Assume that they have a myriadof grocery items at their godown. All items must be listed on the website, along with theirquantities and prices.Users must be able to sign up and purchase groceries. The systemshould present him with delivery slot options, and the user must be able to choose hispreferred slot. Users must then be taken to the payment page where he makes the payment with his favorite method.

This week will have many pages like Header,footer,categories and app.jsx

File Structure:



App.jsx

```
import "./index.css"
import "./App.css"
import products from "./assets/products.json"
import Product from "./components/Product";

export default function App() {
  return (
<div className={"container"}>
<main className={"main"}>
<h1>
        E-Commerce in React and SnipCart
</h1>

<div className={"grid"}>
        {
products.map((product, i) =><Product {...product} key={i}/>)
        }
</div>
</main>
<div
        id="snipcart"
        data-api-
key="NWMwZWNkZGMtZjU2ZS00YzM3LWFlZjYtMmM5Zjk0MWViZDcxNjM3Njg0OTY
0ODg5NTk4MTM3" hidden
>
</div>
</div>
  );
}
```

Components/Product/index.js

```
import "./index.css";

export default function Product(props) {
   const {id, imageUrl, name, description, price} = props

   return (
<div
        key={id}
className={"product"}
>
<img
src={imageUrl}
```

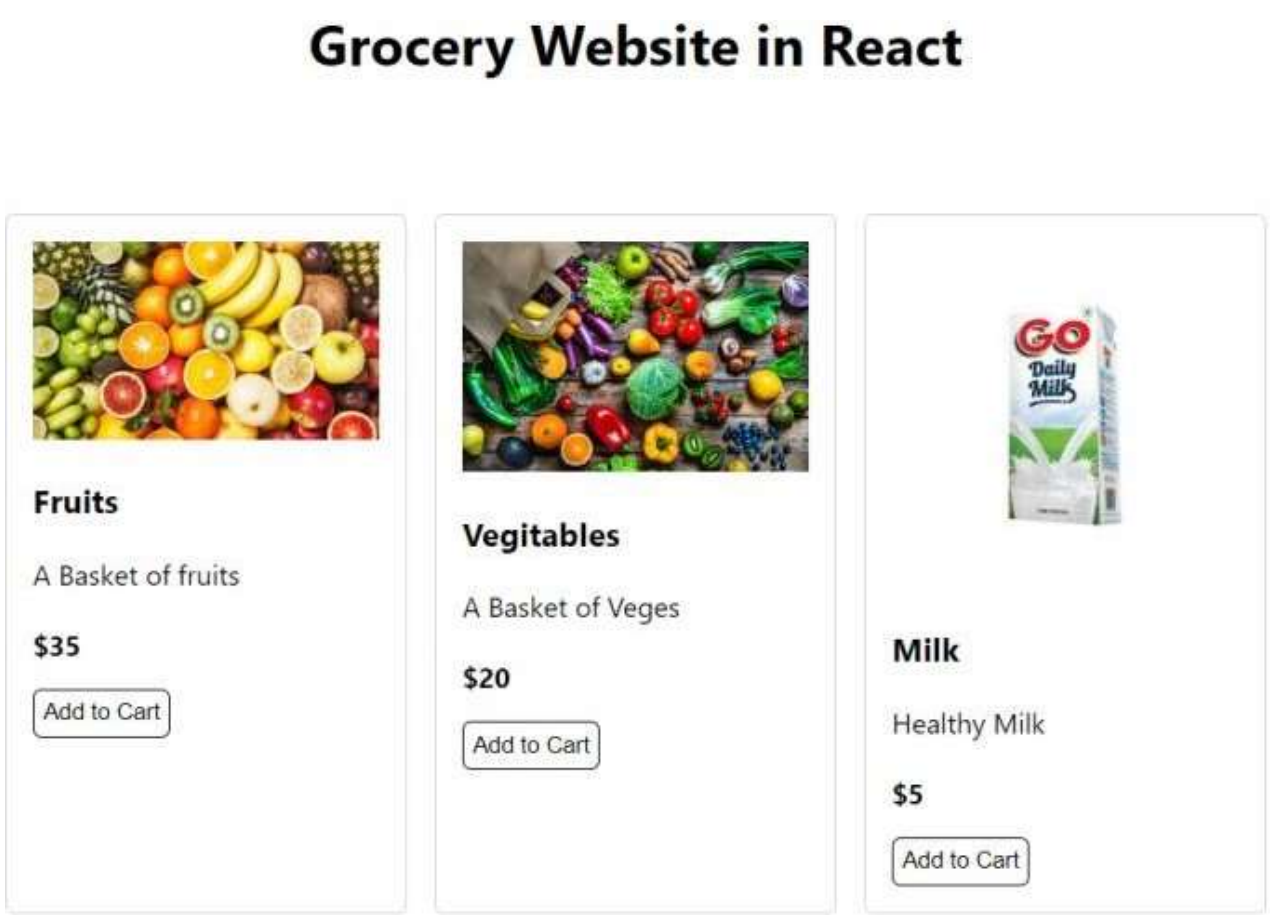
```
            alt={`Image of ${name}`}
className={"image-product"}
          />
<h3>{name}</h3>
<p>{description}</p>
<span>${price}</span>
<div>
<button
className="snipcart-add-item"
                data-item-id={id}
                data-item-image={imageUrl}
                data-item-name={name}
                data-item-url="/"
                data-item-price={price}
>
                Add to Cart
</button>
</div>
</div>
   );

}
```

Assets/products.json

```
[
  {
    "id": "t-shirt",
    "name": "Fruits",
    "price": 35.0,
    "imageUrl":          "https://www.lalpathlabs.com/blog/wp-content/uploads/2019/01/Fruits-and-
Vegetables.jpg",
    "description": "A Basket of fruits",
    "url": "/api/products/halfmoon"
  },
  {
    "id": "wallet",
    "name": "Vegitables",
    "price": 20.0,
    "imageUrl": "https://img.freepik.com/free-photo/bottom-view-fruits-vegetables-radish-cherry-
tomatoes-persimmon-tomatoes-kiwi-cucumber-apples-red-cabbage-parsley-quince-aubergines-
blue-table_140725-146174.jpg",
    "description": "A Basket of Veges",
    "url": "/api/products/wallet"
  },
```

```
  {
    "id": "cup",
    "name": "Milk",
    "price": 5.0,
    "imageUrl":"https://encrypted-
tbn0.gstatic.com/images?q=tbn:ANd9GcSeujHMy6OLRZHTpsrUMVLsHyio1mZiZI4fMQ&us
qp=CAU",
    "description": "Healthy Milk",
    "url": "/api/products/veiltail"
  }
]
```


OUTPUT

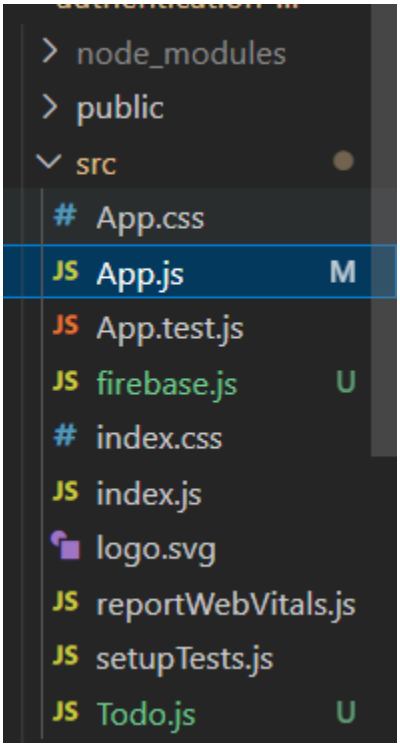


Week-11-12. Connecting our TODO React js Project

We all can create applications but in realtime when we are building an application we have to store the user data some ware now a days best way to store is Firebase which can be integrated in react app

In this week we will learn how to connect our application to firebase

File Structure:



After creating the project make sure to install firebase dependencies:

Install it using npm install firebase

-Now we have mainly 3 pages

1.firebase.js

2.App.js

3.Todo.js

-.In firebase.js we will establish connection to our app and firebase

-In Todo,js we will write the code

And we will import it in to the App.js file

firebase.js

```
import firebase from 'firebase/compat/app';
import 'firebase/compat/auth';
import 'firebase/compat/firestore';

const firebaseApp = firebase.initializeApp({
apiKey: "",
authDomain: "",
projectId:",
storageBucket: ",
messagingSenderId: "",
appId: ":,
measurementId: ""
});

const db = firebaseApp.firestore();

export default db;
```

Note Replace the highlighted code with your firebase connection compoenents

You can get you own keys from firebase account for more details Take the Reference of below video

https://www.youtube.com/watch?v=ad6IavyAHsQ

Todo.js

```
import { ListItem, List, ListItemAvatar, ListItemText, Button, Modal, makeStyles
} from '@material-ui/core'
import './Todo.css';
import React, { useState } from 'react';
import db from './firebase'



function Todo(props) {
   const [open, setOpen] = useState(false);
   const [input, setInput] = useState(props.todo.todo);

   const handleOpen = () => {
```

```
setOpen(true)
   };

   const updateTodo = () => {
      // update to do with the new input text

db.collection('todos').doc(props.todo.id).set({
todo: input
      }, { merge: true })

setOpen(false);
   }

   return (
<>
<div
            open={open}
onClose={e =>setOpen(false)}
>
<div >
<h1>I am a model</h1>
<input placeholder={props.todo.todo} value={input} onChange={event
=>setInput(event.target.value)} />
<button onClick={updateTodo}>Update Todo</button>
</div>
</div>
<ulclassName='todo_list'>
<li>
<li primary={props.todo.todo} secondary='Dummy deadline ⏰' />
</li>
<button onClick={e =>setOpen(true)}>Edit</button>
<button onClick={event
=>db.collection('todos').doc(props.todo.id).delete()}>❌DELETE ME</button>
</ul>
</>
   )
}

export default Todo
```

Now the last file App.js

```
import React, { useEffect, useState } from 'react';
import './App.css';
import Todo from './Todo';
import db from './firebase'
import firebase from 'firebase/compat/app';
import 'firebase/compat/auth';
import 'firebase/compat/firestore';

function App() {
  const [todos, setTodos] = useState([]);
  const [input, setInput] = useState('');

// when the upload, we need to listen to the database and fetch new todos as they get
added/remove
useEffect(() => {
  // This code here... fires when the app.js lodes
db.collection('todos').orderBy('timestamp', 'desc').onSnapshot(snapshot => {
    // console.log(snapshot.docs.map(doc =>doc.data()));
setTodos(snapshot.docs.map(doc => ({id: doc.id, todo: doc.data().todo})))
  })
}, []);

  const addTodo = (event) => {
    // this will fire off when we click the button
event.preventDefault(); //will stop the refresh

db.collection('todos').add({
todo: input,
      timestamp: firebase.firestore.FieldValue.serverTimestamp()
    })

setTodos([...todos, input]);
setInput(' ');  // clear up the input after clicking todo
    console.log(todos)
  }

  return (
<div className="App">
<h1>Build A TODO App 🚀!</h1>

<form>

<form>
```

```
<span>✓ Write a Todo</span>
<input value={input} onChange={event =>setInput(event.target.value)} />
</form>


<button disabled={!input} type='submit' onClick={addTodo} variant="contained" color="primary">Add Todo</button>
</form>

<ul>
      {todos.map(todo => (
<Todotodo={todo}/>
        // <li>{todo}</li>
      ))}

<li></li>
</ul>

</div>
  );
}

export default App;
```

OUTPUT

Installation

VS Code: The steps to install Visual Studio Code (VS Code) on Windows

:1. **Download Visual Studio Code Installer:**

- Go to the official Visual Studio Code website: https://code.visualstudio.com/
- Click on the "Download for Windows" button. This will download the installer (`.exe` file) to your computer.

2. **Run the Installer:**

- Once the download is complete, locate the downloaded `.exe` file (usually in your Downloads folder) and double-click on it to run the installer.

3. **Start Installation:**

- You may see a User Account Control prompt asking for permission to make changes to your device. Click "Yes" to proceed.
- The installer will launch. Click "Next" to begin the installation process.

4. **Accept License Agreement:**

- Read and accept the license agreement by checking the box and clicking "Next".

5. **Choose Installation Options:**

- Choose the destination folder where you want to install Visual Studio Code. The default location is typically ` C:\Program Files\Microsoft VS Code`.
- Optionally, you can choose to add Visual Studio Code to the system PATH, which allows you to run VS Code from the command line.
- Click "Next" to continue.

6. **Select Additional Tasks (Optional):**

- Optionally, you can choose to create shortcuts for Visual Studio Code on the desktop and in the Start Menu.
- Click "Next" to proceed.

7. **Select Start Menu Folder (Optional):**

- Choose the folder where you want to create shortcuts in the Start Menu, or leave it as the default.
- Click "Install" to begin the installation process.

8. **Wait for Installation to Complete:**

- The installer will now extract and install the necessary files. This may take a few moments.

9. **Complete Installation:**

- Once the installation is complete, you will see a "Completing the Visual Studio Code Setup Wizard" screen.
- Ensure that the "Launch Visual Studio Code" option is checked.
- Click "Finish" to exit the installer and launch Visual Studio Code.

10. **Launch Visual Studio Code:**

Node JS: The steps to install Node.js on Windows

1. **Download Node.js Installer:**
   - Go to the official Node.js website: https://nodejs.org/
   - Download the recommended LTS (Long-Term Support) version of Node.js for Windows by clicking on the "LTS" button. This will download the installer (`.msi` file) to your computer.
2. **Run the Installer:**
   - Once the download is complete, locate the downloaded `.msi` file (usually in your Downloads folder) and double-click on it to run the installer.
3. **Start Installation:**
   - You may see a User Account Control prompt asking for permission to make changes to your device. Click "Yes" to proceed.
   - The installer will launch. Click "Next" to begin the installation process.
4. **Accept License Agreement:**
   - Read and accept the license agreement by checking the box and clicking "Next".
5. **Choose Installation Options:**
   - You can choose the destination folder where you want to install Node.js. The default location is typically `C:\Program Files\nodejs`.
   - Optionally, you can customize the components to install. Typically, you don't need to change these settings unless you have specific requirements.
   - Click "Next" to continue.
6. **Select Start Menu Folder (Optional):**
   - Choose the folder where you want to create shortcuts in the Start Menu, or leave it as the default.
   - Click "Next" to proceed.
7. **Wait for Installation to Complete:**
   - The installer will now extract and install the necessary files. This may take a few moments.
8. **Complete Installation:**
   - Once the installation is complete, you will see a "Completing the Node.js Setup Wizard" screen.
   - Ensure that the "Automatically install the necessary tools..." option is checked. This will allow npm (Node Package Manager) to be installed along with Node.js.
   - Click "Finish" to exit the installer.
9. **Verify Installation:**
   - open Command Prompt (or PowerShell) and run the following commands: node -v, npm -v

Express JS: The steps to install Express.js on Windows

To install Express.js, you first need to have Node.js and npm (Node Package Manager) installed on your system, as Express.js is a Node.js framework. Once you have Node.js and npm installed, follow these steps to install Express.js:

1. **Create a New Node.js Project (Optional):**
   - Open your terminal or command prompt.
   - Navigate to the directory where you want to create your new Node.js project. You can use the `cd` command to change directories.
   - If you're starting a new project, you can create a new directory using the `mkdir` command, for example:
     mkdir my-express-app
     cd my-express-app

2. **Initialize Your Node.js Project:**
   - If you're starting a new project, you need to initialize it with a `package.json` file. You can do this by running the following command:
     npm init -y
   - This command will create a `package.json` file with default values.

3. **Install Express.js:**
   - Once your Node.js project is initialized, you can install Express.js as a dependency using npm. Run the following command:
     npm install express
   - This command will download and install Express.js and its dependencies in your project's `node_modules` directory.

4. **Create an Express Application:**
   - After installing Express.js, you can start building your Express application. Create a new JavaScript file (e.g., ` app.js`) in your project directory.

5. **Write Your First Express Application:**
   - In your `app.js` file, require the Express module and create an instance of the Express application:

```
const express = require('express');
const app = express();
```
- Define routes and middleware for your Express application. For example:
```
app.get('/', (req, res) => {
res.send('Hello, Express!');
});
```
- This code defines a route for the root URL (`/`) that sends the text "Hello, Express!" as the response.

6. **Start Your Express Application:**
- Finally, start your Express application by listening on a specific port. Add the following code to your `app.js`
```
const PORT = process.env.PORT || 3000; // Use port 3000 by default
app.listen(PORT, () => {
    console.log(`Server is running on port ${PORT}`);
});
```
- This code starts the Express application and listens for incoming HTTP requests on the specified port (or port 3000 by default).

7. **Run Your Express Application:**
- Save your `app.js` file and run your Express application by executing the following command in your terminal:
```
node app.js
```
- You should see the message "Server is running on port 3000" in the terminal, indicating that your Express application is running.

8. **Access Your Express Application:**
- Open a web browser and navigate to `http://localhost:3000` (or the port you specified). You should see the text "Hello, Express!" displayed in the browser.

That's it! You have successfully installed Express.js and created a basic Express application. You can now continue building and expanding your Express application as needed.

MongoDB Installation Steps: To install MongoDB on Windows, you can follow these steps:

1. **Download MongoDB Installer:**
- Go to the official MongoDB website:
https://www.mongodb.com/try/download/community
  - Choose the version of MongoDB Community Edition suitable for your Windows system. Click on the "Download" button to download the installer.

2. **Run the Installer:**
  - Once the download is complete, locate the downloaded installer file (`.msi` file) and double-click on it to run the installer.

3. **Start Installation:**
  - You may see a User Account Control prompt asking for permission to make changes to your device. Click "Yes" to proceed.
  - The installer will launch. Click "Next" to begin the installation process.

4. **Choose Setup Type:**
  - You will be prompted to choose the setup type. Select "Complete" to install all MongoDB features, including MongoDB Compass (optional).

5. **Choose Installation Folder:**
  - Choose the destination folder where you want to install MongoDB. The default location is typically `C:\Program Files\MongoDB\Server\<version>`.

6. **Install MongoDB as a Service (Optional):**
  - Optionally, you can choose to install MongoDB as a Windows service, which allows MongoDB to start automatically when the system starts. Select the option "Install MongoDB as a Service" and click "Next".

7. **Choose MongoDB Data Directory:**
  - MongoDB requires a data directory to store its data. You can specify the data directory during installation or use the default location (`C:\data\db`).
  - If you choose to specify a different data directory, make sure to create it before proceeding with the installation.

8. **Install MongoDB Compass (Optional):**
   - If you selected the "Complete" setup type, you will be prompted to install MongoDB Compass, a graphical user interface for MongoDB. You can choose to install it or skip it.

9. **Wait for Installation to Complete:**
   - The installer will now extract and install MongoDB and MongoDB Compass (if selected). This may take a few moments.

10. **Complete Installation:**
   - Once the installation is complete, you will see a "Completing the MongoDB Setup Wizard" screen.
   - Click "Finish" to exit the installer.

11. **Verify MongoDB Installation:**
   - After installation, you can verify that MongoDB is installed correctly by opening a command prompt and running the `mongod` command:

     mongod --version

     - This command should display the version of MongoDB installed on your system.

12. **Start MongoDB Service (If Installed as a Service):**
   - If you chose to install MongoDB as a service, it should start automatically. You can also start it manually by running the following command in Command Prompt as an administrator:

     net start MongoDB

To connect MongoDB with Express.js:

1. **Install Required Packages:**

- Before you start, ensure you have the necessary packages installed in your Node.js project. You'll need `express` and `mongoose`. If you haven't installed them yet, you can do so using npm:

```
npm install express mongoose
```

2. **Set Up Express Application:**

- Create an Express application if you haven't already. You can set up a basic Express application in a file named **`app.js`:**

```
const express = require('express');
const app = express();

// Your routes and middleware go here

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

3. **Set Up MongoDB Connection:**

- Import `mongoose` and establish a connection to your MongoDB database. You can do this in the same `app.js` file or in a separate file dedicated to database configuration (`db.js`, for example):

```
const mongoose = require('mongoose');

// Connect to MongoDB
mongoose.connect('mongodb://localhost/my_database', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log('MongoDB connected'))
.catch(err => console.error('MongoDB connection error:', err));
```

4. **Define a Schema:**

- Define a schema for your MongoDB collections. This step is optional but recommended for enforcing a structure for your data. You can define a schema using the `mongoose.Schema` class:

```
const mongoose = require('mongoose');
const Schema = mongoose.Schema;

const userSchema = new Schema({
  name: String,
  email: String,
  age: Number
});

const User = mongoose.model('User', userSchema);
```

5. **Perform CRUD Operations:**

- Now that you've connected to MongoDB and defined a schema, you can perform CRUD (Create, Read, Update, Delete) operations on your collections using Mongoose methods. For example:

```
// Create a new user
const newUser = new User({
  name: 'John Doe',
  email: 'john@example.com',
  age: 30
});
newUser.save();

// Find all users
User.find({}, (err, users) => {
  if (err) throw err;
  console.log(users);
});
```

```
// Update a user
User.findOneAndUpdate({ name: 'John Doe' }, { age: 31 }, { new: true }, (err, user) => {
  if (err) throw err;
  console.log(user);
});

// Delete a user
User.deleteOne({ name: 'John Doe' }, err => {
  if (err) throw err;
  console.log('User deleted successfully');
});
```

6. **Handle Errors and Middleware:**
   - Ensure to handle errors and middleware appropriately in your Express application and database operations.

7. **Testing:**
   - Run your Express application (`node app.js`) and test the connectivity with MongoDB by performing CRUD operations.

These steps should help you set up MongoDB connectivity with Express.js in your Node.js application. Adjust the code according to your specific requirements and project structure.

Interview / Viva Question Answer

These questions and answers should give you a good starting point for preparing for a MERN stack interview.

MongoDB:

1. **What is MongoDB?**
   - MongoDB is a NoSQL database that stores data in flexible, JSON-like documents.
2. **What is BSON?**
   - BSON is a binary representation of JSON-like documents used in MongoDB.
3. **How is data stored in MongoDB?**
   - MongoDB stores data in collections, which contain documents. Documents are composed of field-value pairs and are similar to JSON objects.
4. **What is a replica set in MongoDB?**
   - A replica set is a group of MongoDB servers that maintain the same data set, providing redundancy and high availability.
5. **Explain the difference between MongoDB and SQL databases.**
   - MongoDB is a NoSQL database, which means it does not use structured query language (SQL) for querying data. Instead, it uses a flexible, JSON-like query language.

Express.js:

6. **What is Express.js?**
   - Express.js is a web application framework for Node.js, designed for building web applications and APIs.
7. **What are middleware in Express.js?**
   - Middleware are functions that have access to the request and response objects in an Express application's request-response cycle. They can execute code, modify request and response objects, end the request-response cycle, or call the next middleware function.
8. **How do you handle routing in Express.js?**
   - Routing in Express.js is handled using methods like `app.get()`, `app.post()`, `app.put()`, and `app.delete()`, which correspond to HTTP GET, POST, PUT, and DELETE requests, respectively.
9. **Explain error handling in Express.js.**
   - Error handling in Express.js can be done using middleware functions. Error-handling middleware takes four arguments: `(err, req, res, next)`. If an error occurs, the middleware can generate an error response or pass it to the next error-handling middleware.

10. **What is Express.js generator?**
   - Express generator is a tool used to generate the basic structure of an Express application. It sets up the folder structure and files for an Express application.

React.js:
11. **What is React.js?**
   - React.js is a JavaScript library for building user interfaces, particularly for single-page applications.

12. **What are the key features of React.js?**
   - Key features of React.js include virtual DOM, component-based architecture, JSX syntax, and one-way data flow.

13. **Explain the difference between state and props in React.js.**
   - State is managed within a component and can change over time, while props are read-only and passed from a parent component to a child component.

14. **What is JSX?**
   - JSX (JavaScript XML) is a syntax extension for JavaScript used with React.js to describe what the UI should look like. It allows mixing HTML with JavaScript.

15. **What are React hooks?**
   - React hooks are functions that let you use state and other React features without writing a class. They enable functional components to have state and lifecycle methods.

Node.js:
16. **What is Node.js?**
   - Node.js is a JavaScript runtime built on Chrome's V8 JavaScript engine. It allows developers to run JavaScript code on the server-side.
17. **Explain the event-driven architecture of Node.js.**
   - Node.js is based on an event-driven architecture where certain types of objects (called "emitters") periodically emit named events that cause function objects ("listeners") to be called.

18. **How does Node.js handle concurrent requests?**
   - Node.js is single-threaded but uses an event-driven, non-blocking I/O model to handle concurrent requests efficiently.

19. **What is npm?**
   - npm (Node Package Manager) is the default package manager for Node.js. It allows developers to install, share, and manage dependencies for Node.js projects.

20. **Explain the purpose of package.json in a Node.js project.**
   - package.json is a manifest file that contains metadata about a Node.js project, including its dependencies, scripts, and other project-specific configurations.

General:
21. **What is the MERN stack?**
   - The MERN stack is a full-stack JavaScript framework used for building dynamic web applications. It consists of MongoDB, Express.js, React.js, and Node.js.

22. **What are the advantages of using the MERN stack?**
   - Advantages of using the MERN stack include the ability to use JavaScript throughout the entire application stack, a rich ecosystem of libraries and frameworks, and the flexibility to build both client-side and server-side applications.

23. **Explain the role of each component in the MERN stack.**
   - MongoDB: Database layer for storing data.
   - Express.js: Web application framework for Node.js, handling routing and middleware.
   - React.js: JavaScript library for building user interfaces.
   - Node.js: JavaScript runtime for server-side code execution.
24. **How do you secure a MERN stack application?**
   - Securing a MERN stack application involves implementing measures such as authentication, authorization, input validation, encryption, and using HTTPS for communication.
25. **What tools can be used for debugging in the MERN stack?**
   - Tools like Chrome Developer Tools, Redux DevTools (for React), Postman (for API testing), and Node.js debugger can be used for debugging in the MERN stack.